PRESENTATION SCRIPT · 5 MINUTES · 10 SLIDES

Ember — what to say on every slide

Short sentences, one idea per slide, and a memory trick for the order. Learn the coloured words — the rest of the sentence comes naturally.

Remember the order with one sentence

Please Serve All Eleven Kebabs — Smoky, Delicious, Crispy.

Problem · Solution · Architecture · ER (eleven tables) · Keys · SQL · Demo · Conclusion

The whole talk at a glance

Slide	Title	Remember	Time	Clock
1	Title	Name · Guide · Ember = one MySQL database	10 s	0:10
2	Problem & Abstract	Paper → 4 problems · 1 database · 11 tables	30 s	0:40
3	Proposed Solution	Two front-ends · one database in the middle	30 s	1:10
4	System Architecture	Browser → PHP → MySQL → HTML back	30 s	1:40
5	ER Diagram & Schema	Accounts on top · 1:1 · 1:N · M:N → Bill_Items · 11 tables	40 s	2:20
6	Keys, Constraints & Normalization	PK · FK · stored once in Accounts	40 s	3:00
7	SQL in Action	NOT IN · JOIN + SUM · COMMIT / ROLLBACK · ?	40 s	3:40
8	Demo Screens	POS · Your Order · Home	30 s	4:10
9	Conclusion & Future Work	1 database in sync · concepts live · hash · pay · views	30 s	4:40
10	Thank You	Thanks · questions · offer demo	10 s	4:50

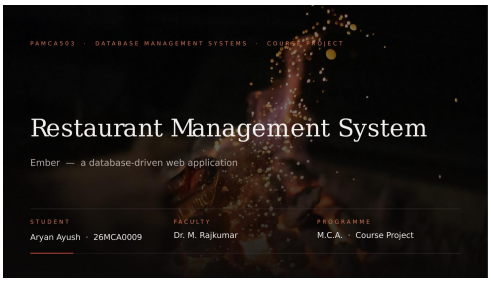
The slide changes take about 1 second each (the Morph transition), so the full talk lands just under 5 minutes.

Three things to do every time

– **Click, wait for the transition, then speak.** Your name glides to the corner — let it land before the next sentence.

– **Point at one thing per slide** (it is written under each picture). It keeps you calm and the faculty looking where you want.

– **Say the bridge line** at the end of each slide. It makes the talk sound like one story, not ten separate slides.



Point at: *Stand still, smile, then click.*

SLIDE 1 — 10 s

Title

REMEMBER: Name · Guide · Ember = one MySQL database

Good morning. I'm Aryan Ayush, 26MCA0009. My DBMS course project, guided by Dr. M. Rajkumar, is Ember — a restaurant management system built on one MySQL database.

→ *Click, then say:* "Let me start with the problem."



Point at: *Count the four problems on your fingers. Point at the big "11".*

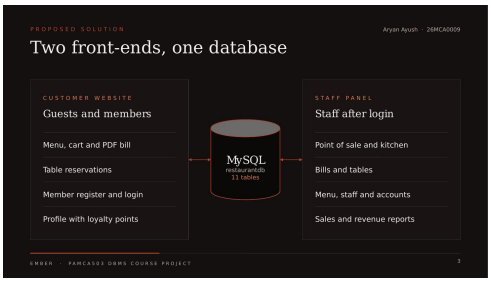
SLIDE 2 — 30 s

Problem & Abstract

REMEMBER: Paper → 4 problems · 1 database · 11 tables

Many restaurants still run on paper registers. That causes four problems: tables get double-booked, orders get lost, bills have mistakes, and there is no history of members or sales. My answer is simple: one MySQL database holds the whole restaurant — eleven related tables. A customer website and a staff panel use the same tables, so every screen always agrees.

→ *Click, then say:* "So what did I build?"



Point at: *Left card, right card, then the cylinder in the middle.*

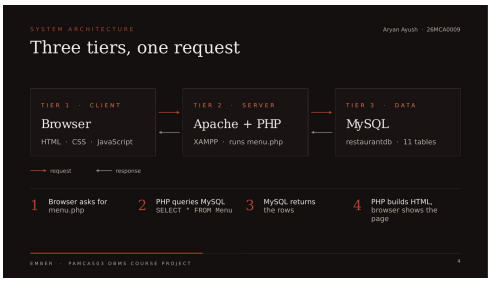
SLIDE 3 — 30 s

Proposed Solution

REMEMBER: Two front-ends · one database in the middle

Two front-ends, one database. On the left, the customer website: menu, cart and PDF bill, table reservations, and member login with loyalty points. On the right, the staff panel: point of sale and kitchen, bills and tables, menu and staff, and sales reports. In the middle is MySQL — both sides read and write the same tables.

→ *Click, then say:* "Here is how one request travels."



Point at: *Trace the arrows left to right, then the four numbered steps.*

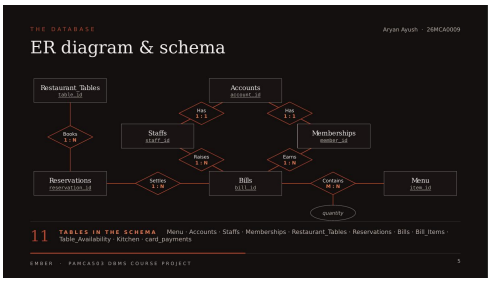
SLIDE 4 — 30 s

System Architecture

REMEMBER: Browser → PHP → MySQL → HTML back

Ember has three tiers: the browser, Apache with PHP inside XAMPP, and MySQL. When you open the menu, the browser asks for menu.php. PHP runs a SELECT on the Menu table, MySQL returns the rows, and PHP turns them into HTML. The browser never touches the database directly — that keeps the data safe.

→ *Click, then say:* "Now the heart of the project — the database."



Point at: *Top box first, then the "Contains M : N" diamond, then the list of 11 tables.*

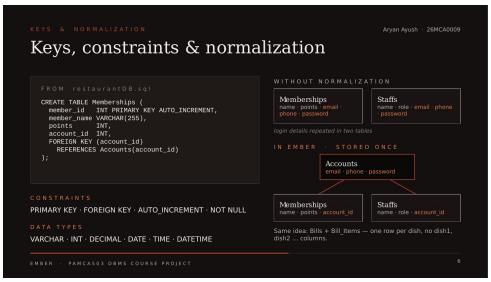
SLIDE 5 — 40 s

ER Diagram & Schema

REMEMBER: Accounts on top · 1:1 · 1:N · M:N → Bill_Items · 11 tables

This is the ER diagram. At the top is Accounts — every staff member and every member has exactly one account. Staff raise bills and members earn points on bills — one-to-many. A bill contains many dishes and a dish is on many bills — that is many-to-many, so it becomes the Bill_Items table with a quantity. Each reservation books one table. In total, eleven tables, linked by primary and foreign keys.

→ *Click, then say:* "How do we keep these tables correct?"



Point at: *Code card on the left, then "Without" → "In Ember" on the right.*

SLIDE 6

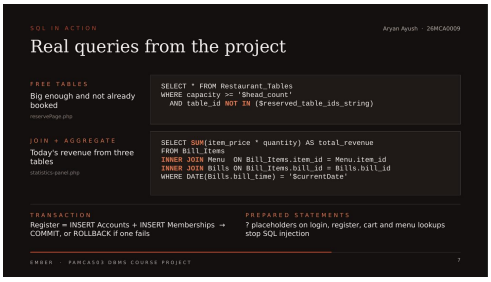
40 s

Keys, Constraints & Normalization

REMEMBER: PK · FK · stored once in Accounts

With keys and constraints. Every table has a primary key, usually AUTO_INCREMENT. Foreign keys link the tables — here Memberships.account_id must match a real account. For normalization: without it, Memberships and Staffs would each repeat email, phone and password. In Ember those are stored once in Accounts, and both tables just point to it. Bills and Bill_Items follow the same idea. No duplicate data, no update anomalies.

→ *Click, then say:* "And here is SQL actually working."



Point at: *Point at the orange words: NOT IN, SUM, INNER JOIN.*

SLIDE 7

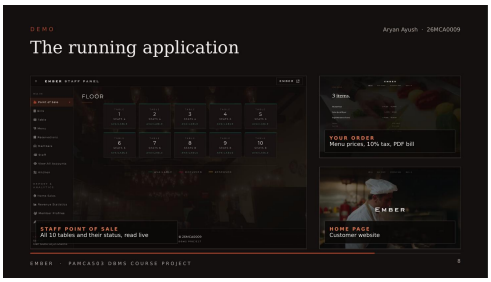
40 s

SQL in Action

REMEMBER: NOT IN · JOIN + SUM · COMMIT / ROLLBACK · ?

Two real queries. First, free tables: tables with enough capacity whose ID is NOT IN the list already booked at that time. Second, revenue: it joins three tables — Bill_Items, Menu and Bills — and uses SUM of price times quantity. Registration is a transaction: insert the account and the membership, then COMMIT — or ROLLBACK if one fails. And logins, registration and the cart use prepared statements against SQL injection.

→ *Click, then say:* "Let me show you it running."



Point at: *Big screen first, then top right, then bottom right.*

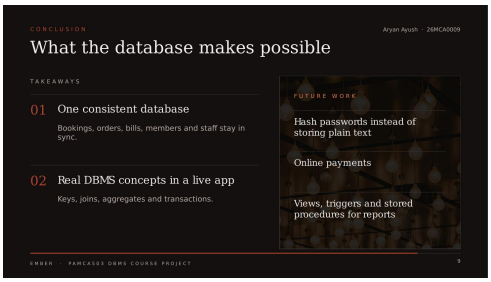
SLIDE 8 — 30 s

Demo Screens

REMEMBER: POS · Your Order · Home

These are real screens. The big one is the staff point of sale — all ten tables and their status, read live from the database. Top right is Your Order: prices come from the Menu table, ten percent tax is added, and the bill downloads as a PDF. Bottom right is the customer home page. If there is time, I can show it live.

→ *Click, then say:* "To wrap up..."



Point at: *Left side for takeaways, the photo box for future work.*

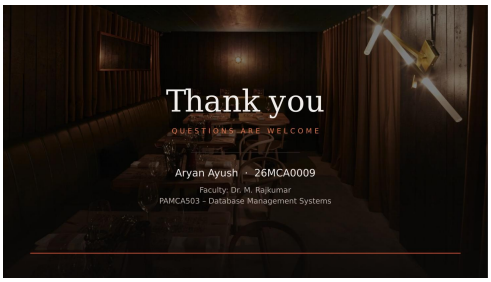
SLIDE 9 — 30 s

Conclusion & Future Work

REMEMBER: 1 database in sync · concepts live · hash · pay · views

Two takeaways. First, one consistent database runs the whole restaurant — bookings, orders, bills, members and staff stay in sync. Second, it shows real DBMS concepts in a live app: keys, joins, aggregates and transactions. For future work I would hash passwords instead of storing plain text, add online payments, and use views, triggers and stored procedures for reports.

→ *Click, then say:* "Thank you."



Point at: *Pause. Look at the faculty, not the screen.*

SLIDE 10 — 10 s

Thank You

REMEMBER: Thanks · questions · offer demo

Thank you for listening. I am happy to take questions, and I can also show a live demo of the website and the staff panel.

AFTER THE TALK

Likely questions — short answers

Answer in one or two sentences, then stop. If you don't know, say what you would check in the code.

Why did you use a DBMS instead of files?
Keys and constraints keep data correct, many users can work at once, SQL answers questions, and transactions protect data.

What is a primary key and a foreign key in your project?
Bills.bill_id is a primary key. Bill_Items.bill_id is a foreign key that must match a real bill.

Which normal form is your database in?
The main tables follow 3NF ideas — each fact is stored once, like login details in Accounts. A few tables, like Bills, keep extra links for convenience.

How do you stop SQL injection?
Prepared statements: the query uses ? placeholders and the values are sent separately, so they can never become SQL.

Where is the cart stored and why?
In the PHP session, not the database. It is only a draft for one visitor; real orders are made by staff in the POS.

Passwords are plain text — is that safe?
No, and I listed it as future work: hash with password_hash() and check with password_verify().

How are member points given?
When a bill with a member is paid: UPDATE Memberships SET points = points + total. From 1000 points the member is VIP.

How does the staff panel stop customers?
Every staff page includes checkIfLoggedIn.php. If there is no staff login in the session, it redirects and stops with exit.

How to learn this in one evening

Round	What to do	Goal
1	Read the whole script out loud with the slides open.	Hear how it flows
2	Read only the REMEMBER lines and say the rest in your own words.	Stop memorising sentences
3	Use only the kebab sentence and the slides. Time yourself.	Under 5:00
4	Answer the questions above without looking.	Calm in the viva